Applications

Appliances

Metering

Robotics & Automation

Healthcare

Power Tools

Secure Locks

Surveillance

Smart Homes & Buildings

Drives, Pumps, Compressors

Lighting

Verification

Overview – Why, What & How

1 Why to Verify?

- **Catch Bugs Early**
  RTL bugs found late cost 100x more; verification prevents re-spins
- **Ensure Spec Compliance**
  Confirms design meets all protocol and architectural specifications
- **Reduce Silicon Risk**
  Avoids costly tape-out failures and missed market windows
- **Improve Design Quality**
  Builds confidence across functional, power and timing dimensions
- **Enable Safe IP Reuse**
  Verified IPs can be safely integrated into future SoC designs

2 What to Verify?

- **Functional Verification**
  Design behaves per spec across all scenarios and corner cases
- **Performance Verification**
  Throughput, latency and timing validated under real workloads
- **Power Intent Verification**
  Power domains, UPF rules and low-power modes are correct
- **Gate Level Verification**
  Post-synthesis simulation with SDF back-annotation

3 How to Verify?

- **Simulation-based**
  RTL simulation using UVM / SystemVerilog testbenches
- **Formal & Semi-formal**
  Exhaustive property checking; bounded model checking
- **Emulation / FPGA**
  Hardware-accelerated verification at near real-speed
- **HW/SW Co-Verification**
  Joint validation of hardware and firmware together

Key Takeaway

1. Verification prevents costly silicon re-spins and ensures market-ready chips
2. Combine simulation, formal & emulation for complete SoC coverage confidence
3. Power & Gate-level checks are mandatory pre-tape-out milestones

What is Verification?

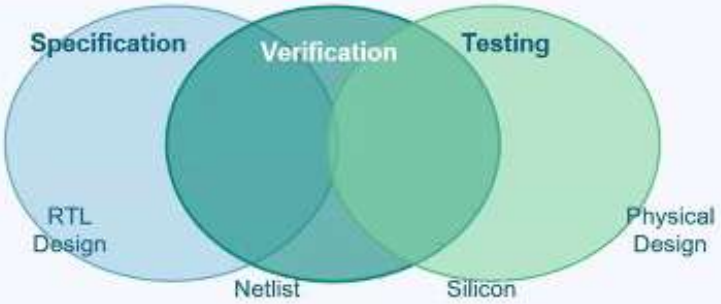
Verification is the process of ensuring that a design correctly implements its intended specification. In ASIC/SoC development, it is also known as **Design Verification (DV).**

✓ Verification answers the question: "Did we design it right?"

Purpose of Verification

- To check functional correctness of RTL
- To ensure the design matches the specification
- To detect bugs before silicon fabrication
- To validate corner cases and boundary conditions
- To reduce costly silicon re-spins

Verification Vs Testing Vs Validation



Verification	Testing	Validation
✓ Ensures RTL matches specification	✓ Done after silicon fabrication	✓ Ensures product meets customer / system requirements
✓ Uses simulation, formal, assertions, coverage	✓ Checks manufactured chip functionality	

Functional Verification

iPhone Use Case

Functional Verification – iPhone Use Case

What is Functional Verification?

Ensuring the iPhone works correctly under normal, stress, multi-app and corner-case conditions.

Key Features to Verify:

- **Multi-Application Concurrency –**
  WhatsApp + Instagram + Teams, rapid switching, memory pressure handling
- **Calling & Communication –**
  Call waiting, VoIP ↔ Cellular switching, network drop during call, Bluetooth switching
- **Messaging & Notifications –**
  Notification priority, DND behavior, lock screen privacy, notification flood handling
- **Performance & Stress –**
  Gaming + Call, video playback + background apps, thermal throttling, battery drain, WiFi ↔ LTE handover

★ **Key Takeaway:** Functional Verification means testing real-world concurrent scenarios to hunt deep-rooted bugs.

Functional Verification

- **Functional testing is one approach to ensuring quality**
- **Quality refers to the state of the product**
  Where it performs as intended under various operating conditions
- **Basic feature bugs are detected with basic test cases**
- **Deep rooted bugs are difficult to find out**
  Need multiple combinations of inputs to expose them
- **As features and complexity increases**
  The chances of complex bugs also increase

iPhone Use Case

Why it matters:

- Each iOS version adds 100s of features
- Bugs in calls/apps = user frustration
- Verification ensures release quality

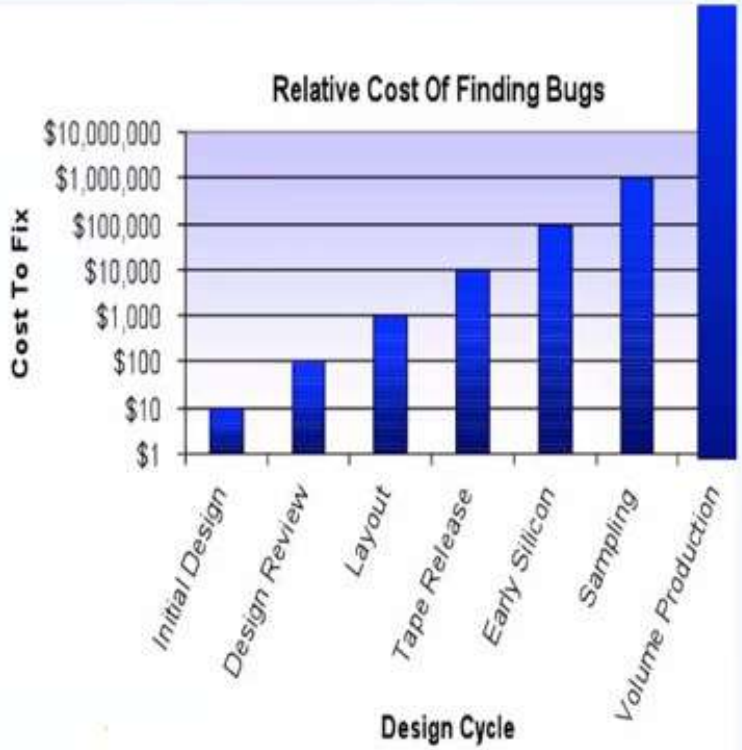
Why Verification?

Cost of Finding Bugs

Reasons to Verify

- Ensure design implements intended functionality
- Detect logical and corner-case bugs early
- Prevent costly silicon re-spins
- Reduce post-silicon debug effort
- Improve design robustness and reliability
- Validate multi-IP integration and concurrency
- Increase tape-out confidence with coverage metrics
- Protect company reputation — avoid field failures
- Meet time-to-market and compliance deadlines
- Enable safe IP reuse across future SoC designs

Relative Cost of Finding Bugs — Design Cycle

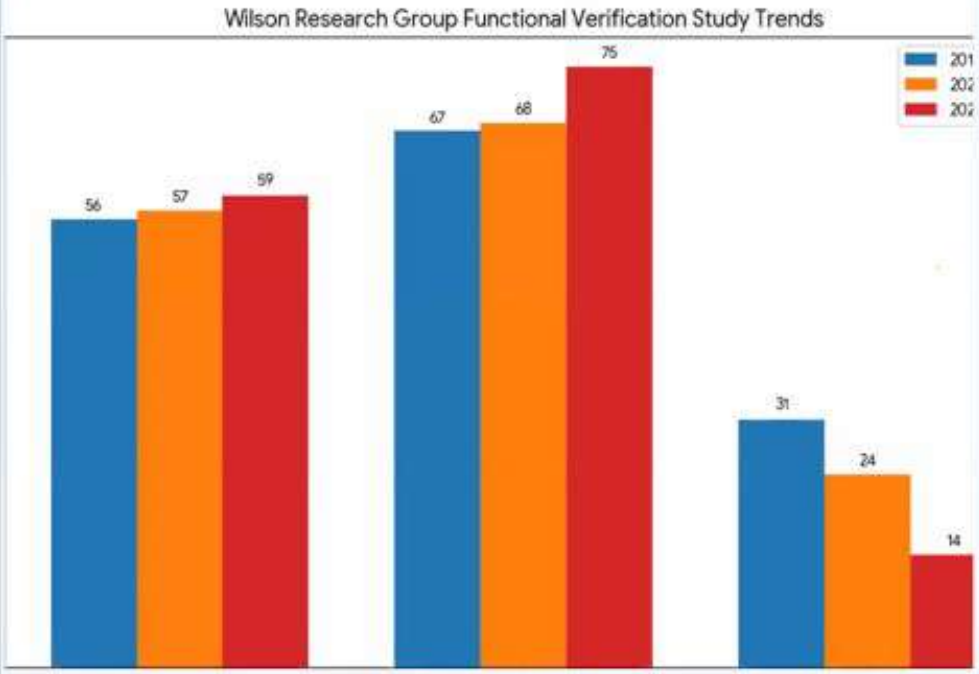


Source: Silicon Debug, Doug Josephson & Bob Gottlieb (Paul Ryan); D. Gizopoulos (ed.), Advances in Electronic Testing, Springer, 2006

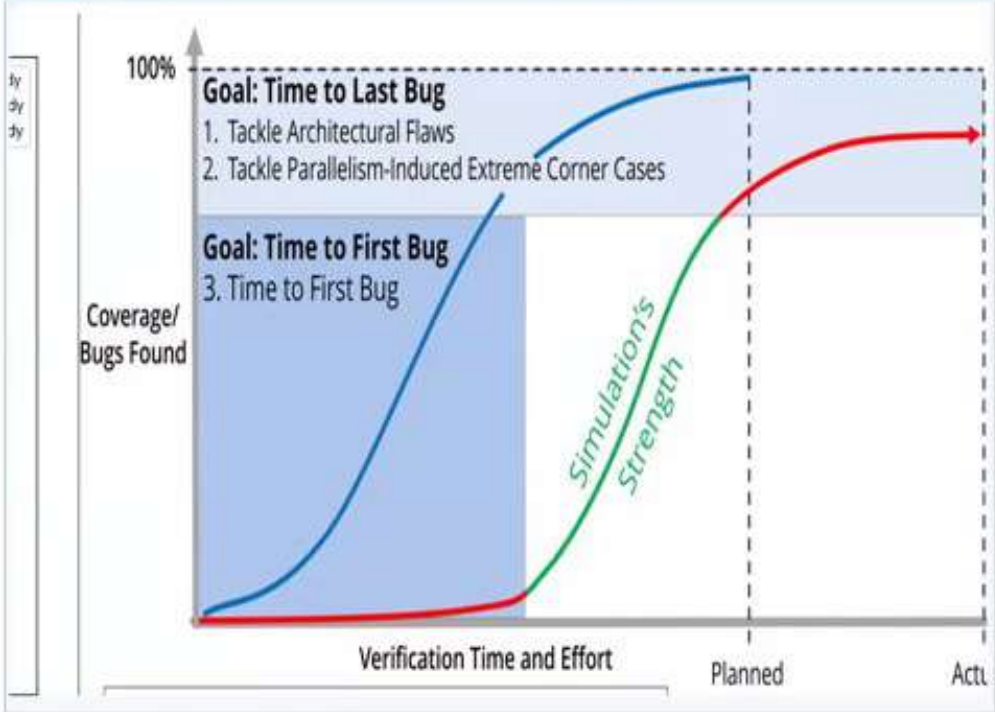
Verification Time

Industry Trends & Coverage Goals

Wilson Research Group — Functional Verification Study Trends



Coverage Goals vs. Verification Time & Effort



Key Insights

1. 75% of projects are behind schedule — verification bottleneck is real (2024 study)
2. First-silicon success improved to 86% — thanks to better verification coverage
3. Simulation + Formal combined finds bugs faster than simulation alone (NEW)
4. Verification now consumes ~59% of total project time — plan early (NEW)

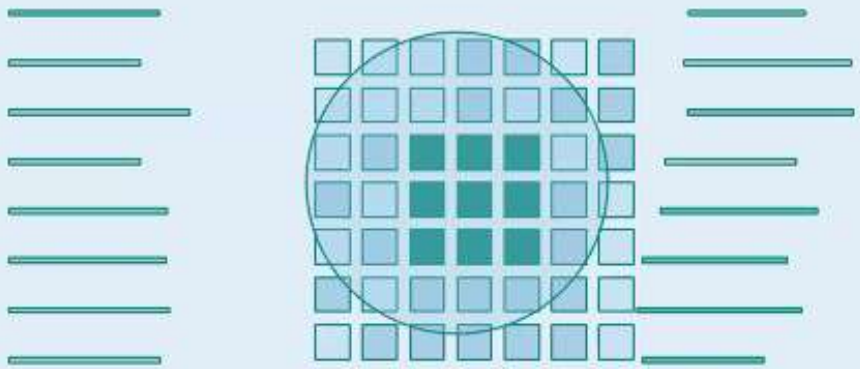
How Do We Do Verification?

1 Understand the Design

- Study and understand the RTL specification
- Identify features and corner cases
- Prepare the **Verification Plan**
- Review with designer / architect

2 Plan the Testbench Architecture

- Identify required components
  - Driver
  - Monitor
  - Scoreboard
  - Reference Model
  - Coverage
  - Coverage
- Define stimulus and checking strategy



RTL Design

How Do We Do Verification?

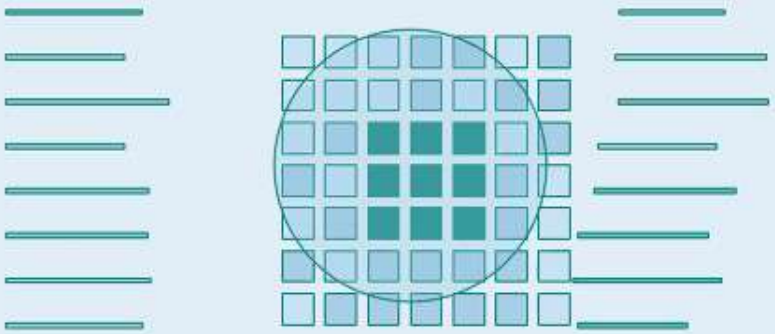
Part 1 – Planning & Setup

1 Understand the Design

- Study and understand the RTL specification
- Identify features and corner cases
- Prepare the **Verification Plan**
- Review with designer / architect

2 Plan the Testbench Architecture

- Identify required components
  - Driver
  - Monitor
  - Scoreboard
  - Reference Model
  - Coverage
  - Coverage
- Define stimulus and checking strategy



RTL Design

How Do We Do Verification?

Part 2 – Execution & Sign-Off

4 Develop Tests

- Write tests as per verification plan
- Cover normal, corner, and error scenarios
- Add constraints and assertions

5 Run & Analyze

- Execute regressions
- Debug failures
- Track coverage (Code + Functional)
- Close coverage gaps

3 Implement the Environment

- Code testbench using SystemVerilog / UVM
- Connect all components
- Run basic sanity test

6 Sign-Off

- Achieve coverage targets
- All critical scenarios verified
- Bugs resolved
- Design ready for tape-out

Design Under Verification

DUV / DUT

Design Under Verification (DUV)

Any RTL design which needs to be verified is called a DUV

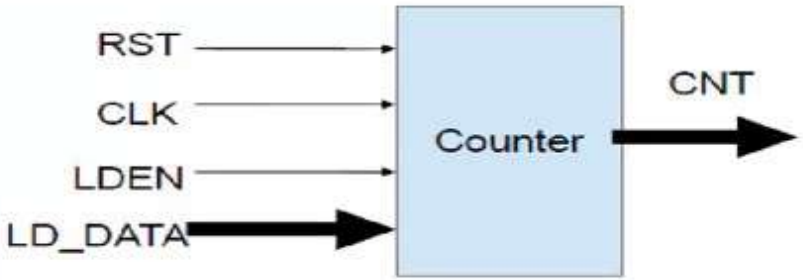
Also Known As Design Under Test (DUT)

Both terms are used interchangeably in the industry

The Golden Reference

The spec defines expected behavior; DUV must match it exactly

Counter DUV — Block Diagram



If, upon asserting RST and LDEN at the same time, CNT does not become 0, then this is a bug and it is due to incorrect coding as shown

Verilog Code — Counter Module (with Priority Bug)

```
// Verilog code for counter

module COUNTER (RST,CLK,CNT,LDEN, LD_DATA);

  input RST,CLK,LDEN;
  input [2:0] LD_DATA
  output reg [2:0] CNT;

  always @ (posedge CLK )
    if (LDEN)
        CNT <= LD_DATA;
    else if(RST)
        CNT <= 0;
    else
   CNT <= CNT + 1;

endmodule
```

LDEN has Higher priority than RST

Sanity Check

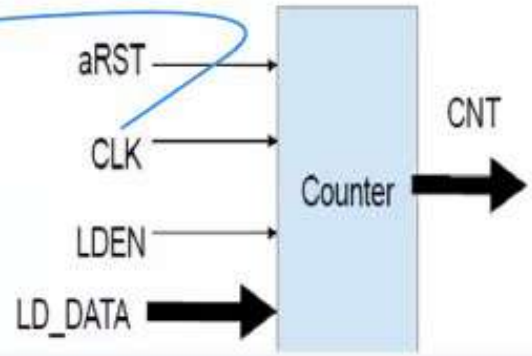
RTL Integrity Verification

What is a Sanity Check?

A Sanity Check is a basic integrity check of an RTL design — the first test run before full regression to ensure the testbench and design are fundamentally working.

1. **Compilation Errors**
   Ensure RTL compiles without syntax or elaboration errors
2. **Reset Check**
   No "unknowns" (X) should be present in the design after reset
3. **Basic Connectivity**
   DUT ports are connected correctly to testbench interfaces
4. **Clock & Reset Propagation**
   Clock toggles and reset deasserts as expected
5. **Initial State Verification**
   Output signals are in known, defined states post-reset

Counter DUV — Sanity Check Example



For example:
Is CNT = 0 upon assertion aRST(asynchronous reset) signal?

Sanity Check Questions:

- Is CNT = 0 upon assertion of aRST (asynchronous reset)?
- Does CLK toggle without simulation hang?
- Are all ports driven — no floating X values?
- Does design elaborate without warnings?

RTL Verification

Exhaustive Feature Che

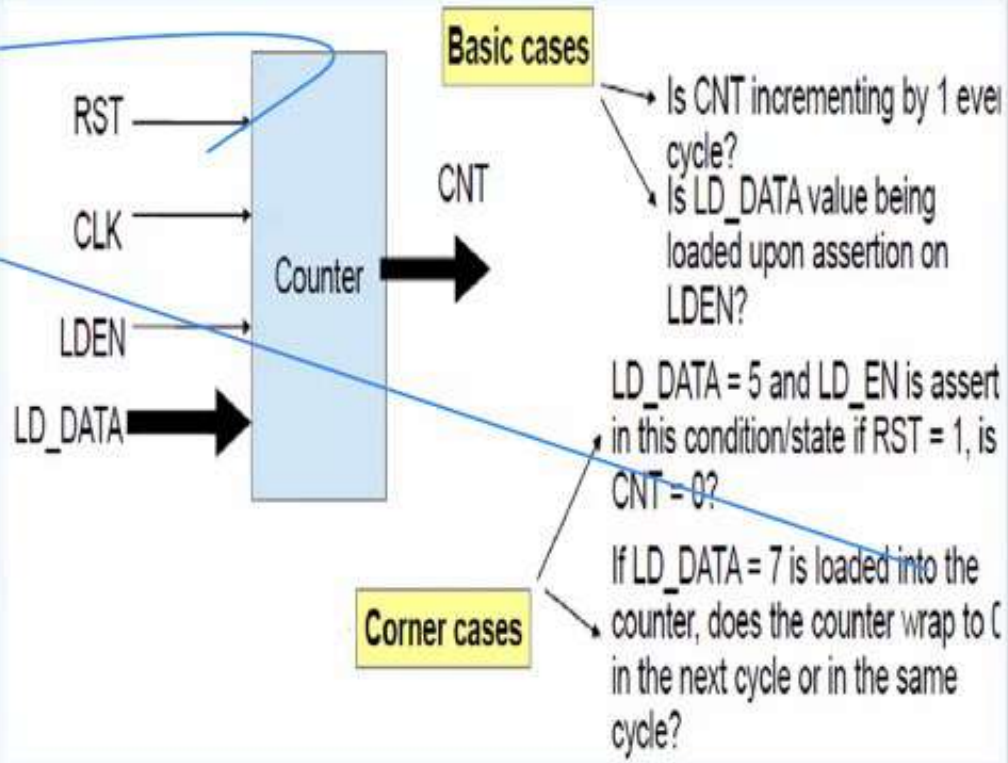
Talking: Priy

RTL Verification is defined as exhaustive checking of the RTL design for every feature starting from every state. It is performed using a **testbench.**

Test Categories — Counter DUV

- **Basic Cases**
  - Is CNT incrementing by 1 every clock cycle?
  - Is LD_DATA value being loaded upon assertion on LDEN?
  - Does RST drive CNT to 0 correctly?
- **Corner Cases**
  - LD_DATA = 5, LD_EN asserted & RST = 1 simultaneously — is CNT = 0?
  - If LD_DATA = 7 loaded, does counter wrap to 0 in next or same cycle?
  - What happens if CLK is held low for multiple cycles?

Counter DUV — Test Case Diagram



RTL Testbench

Self-Checking Infrastructure

An RTL Testbench is the infrastructure needed to run RTL verification. It is a **separate entity from RTL design** — well structured, modular, and written in an HDL/HVL (e.g. SystemVerilog). Testbenches meeting all objectives are called **self-checking testbenches.**

Testbench — Key Characteristics

1. **Infrastructure for RTL Verification**
   Provides the complete environment — stimulus, checking, coverage — to simulate and verify the RTL design
2. **Separate Entity from RTL Design**
   Testbench code is never synthesised — it exists only for simulation and must not be part of the design netlist
3. **Well Structured & Modular**
   Built with well-defined functional blocks (Driver, Monitor, Scoreboard) and written in HDL/HVL such as SystemVerilog
4. **Self-Checking Testbench**
   Automatically compares actual DUT output against expected results — no manual waveform inspection needed
5. **Reusable & Parameterised**
   Modern testbenches (UVM) are parameterised and reusable across multiple DUT configs and protocol variants

Testbench Objectives — Self-Checking

① **Drive Stimulus** — Driver
Apply input test vectors to the RTL design inputs (RST, CLK, LDEN, LD_DATA, etc.)

② **Monitor Response** — Monitor
Observe and capture output signals at the DUT output (e.g. CNT[2:0]) after each stimulus

③ **Automatically Check Response** — Scoreboard
Compare actual output against expected results — report PASS or FAIL without manual intervention

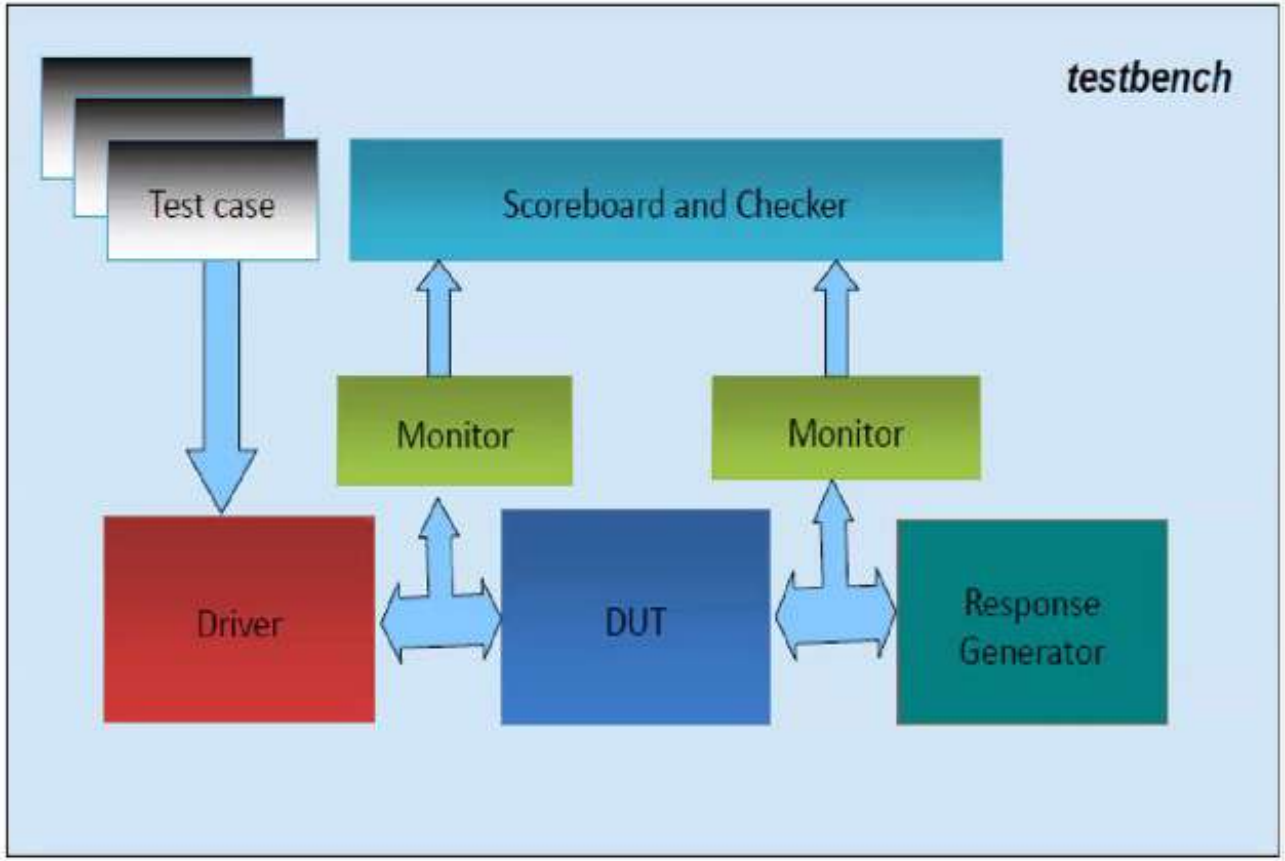
④ **Maintain Results & Coverage** — Coverage
Log simulation results, functional coverage hits, and code coverage metrics per test run

⑤ **Create Reports** — Reporter
Generate simulation log, coverage report, and regression summary for sign-off review

✓ **Self-Checking TB = Objectives ①②③④⑤ all implemented in one testbench**

Typical Functional Blocks of a Self-Checking Testbench

Testbench Architecture Diagram



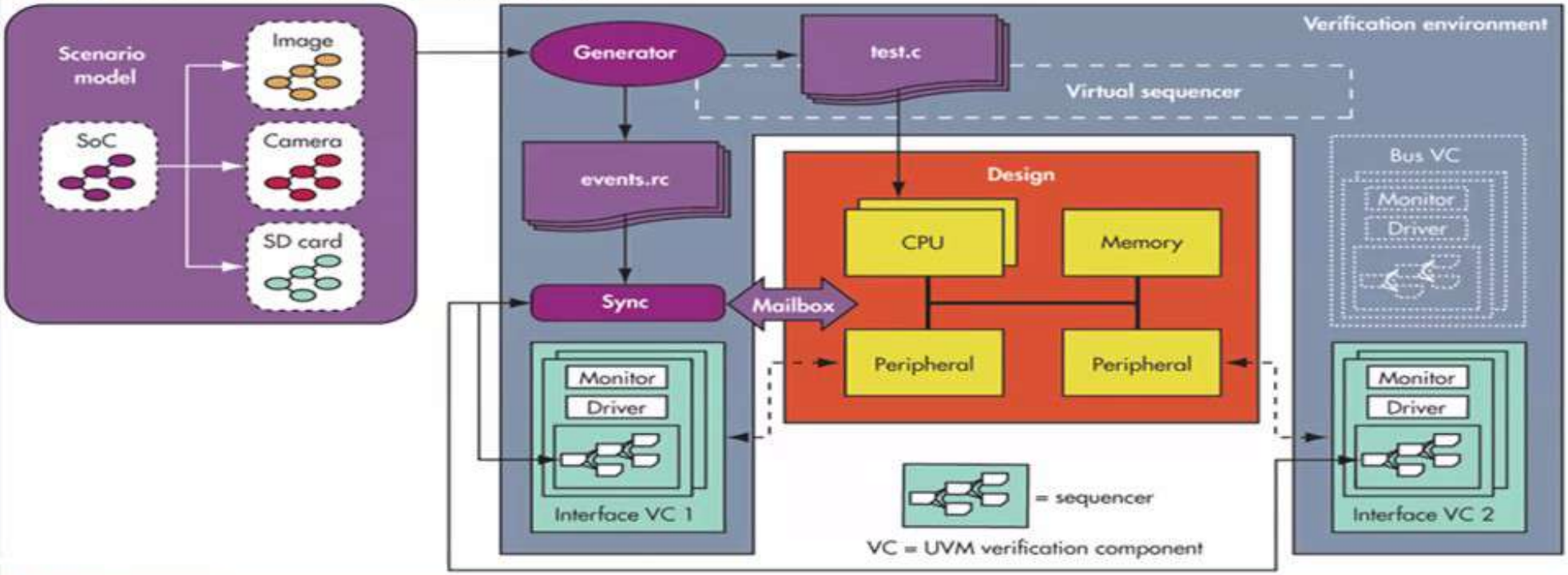
Block Descriptions

- **Driver**
  Applies stimulus to DUT inputs based on the test case
- **DUT**
  Design Under Test — the RTL module being verified
- **Monitor**
  Observes DUT ports and captures transactions for checking
- **Response Generator**
  Generates expected outputs for comparison
- **Scoreboard & Checker**
  Compares actual vs expected output; flags mismatches
- **Test Case**
  Defines stimulus scenarios — basic, corner, and error cases

Verification Testbench

UVM Architecture Overview

UVM Verification Environment — Block Diagram



Key Components

Scenario Model
Defines realistic usage scenarios (SoC, Camera, SD Card)

Generator
Converts scenarios into test stimuli (test.c, events.rc)

Virtual Sequencer
Coordinates multiple VCs via virtual sequences

Design (DUT)
SoC design: CPU, Memory, and Peripherals

Interface VC
Monitor + Driver + Sequencer per interface

Bus VC
Verification component for bus-level checking

Design Under Verification

DUV / DUT — Testbench Integration

DUV (Design Under Verification) is any RTL design which needs to be verified. Also referred to as **DUT (Design Under Test).** The DUV is instantiated inside the top-level testbench alongside the Driver and Monitor.

DUV — Key Points

1. **What is DUV?**
   Any RTL design that needs to be verified — could be a module, IP block, or full SoC
2. **Also Called DUT**
   Both terms (DUV and DUT) are used interchangeably across the industry
3. **Instantiated in Testbench**
   The DUV is instantiated in the top-level testbench module alongside Driver and Monitor
4. **Port Connections**
   DUV ports (.RST, .CLK, .COUNT) are connected to shared testbench signals (rst, clk, cnt)
5. **Black-Box Perspective**
   The testbench treats the DUV as a black box — only inputs/outputs matter, not internal logic
6. **Isolation from Testbench**
   Interfaces and virtual interfaces are used in UVM to cleanly separate DUV from verification env

Verilog — Top-Level Testbench with DUV Instantiation

```
// Top level integration of DUT, driver and

module test_bench();

driver drv ( .clk(clk),
             .rst(rst) );

COUNTER duv( .RST(rst),
             .CLK(clk),
             .COUNT(cnt) );

Monitor monitor (.rst(rst),
                 .clock(clk),
                 .count(cnt) );
```

Stimulus

DUT Input Data

Stimulus is the data applied to the input of the DUT to perform simulations to verify its functionality. It can be instantaneous or time-varying depending on the nature of the DUT and verification strategy.

Types of Stimulus

① Instantaneous Stimulus

Applied at a single point in time — the value is driven once and observed immediately.

- *Example: Loading data into the counter (LDEN + LD_DATA applied for 1 cycle)*

② Time-Varying Stimulus

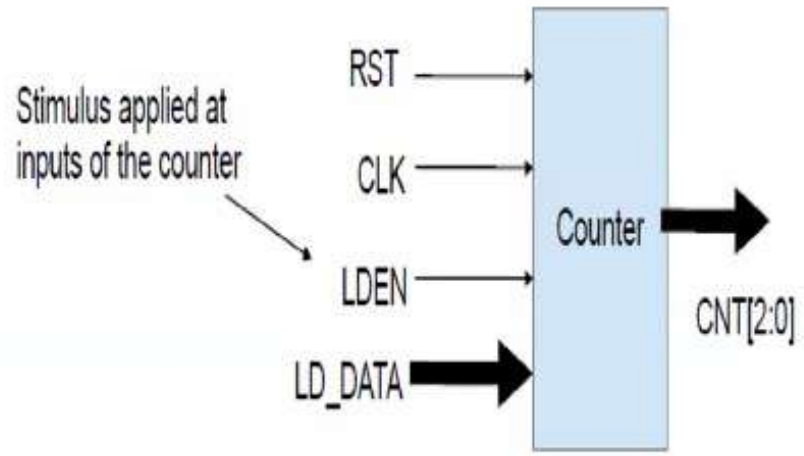
Applied over multiple clock cycles — values change over time based on protocol or sequence.

- *Example: Loading serial data into a SISO shift register (1 bit per cycle)*

③ Constrained-Random Stimulus

- *Example: SystemVerilog rand variables with constraints to auto-generate diverse test vectors*

Counter DUV — Stimulus Applied at Inputs



RST	CLK	LDEN	LD_DATA
Instantaneous	**Time-Varying**	**Instantaneous**	**Instantaneous**
Assert once to reset counter	Continuous periodic toggle	1-cycle pulse to load data	Parallel data loaded with LDEN

What is a Verification Plan?

VPlan / Testplan

A Verification Plan is a specification document for verification — the mechanism to ensure all essential features are verified by defining **What, How, and When** each feature will be checked.

Why is the Verification Plan Important?

1. **Documents Design Spec Interpretation**
   Records how the verification team interprets the design spec — prevents misalignment with the designer
2. **Effort Estimation & Planning**
   Helps in effort estimation and scheduling — sets realistic timelines for each phase
3. **Defines Ownership & Deliverables**
   Assigns responsibility for each feature/test — clear ownership prevents coverage gaps
4. **Supports Formal Review Process**
   Used as the basis for verification review meetings with designers and architects
5. **Staged Development with Design Progress**
   Evolves alongside design — updated as RTL matures from block level to full chip
6. **Coverage Closure Tracking (NEW)**
   Maps each planned feature to functional coverage points — tracks sign-off progress

Verification Plan — What, How & When

WHAT	HOW	WHEN
Features to verify	Simulation / Formal / Emulation	Phase: block vs chip level
Corner cases & error scenarios	Directed vs constrained-random	Milestone tied to RTL freeze
Protocol compliance checks	Assertion-based checks	Sign-off before tape-out

Typical Sections in a Verification Plan

- Design Overview & Scope
- Feature List with Priority
- Testbench Architecture
- Coverage Plan & Metrics
- Schedule & Milestones
- Sign-Off Criteria

Verification Plan — Contents

What · How · When

① What to Verify?

Define the complete scope of verification

Features & Configurations
All functional modes the design must support

Scenarios & Reset / Boot
Power-on, reset sequences, boot/initialization flows

Stress Scenarios
Back-pressure, overflow, simultaneous events

Usage Models
Real-world usage patterns from product spec

Performance & Power
Throughput, latency, power-state transitions

Error Injection
Bus errors, protocol violations, illegal inputs

② How to Verify?

Define verification strategy and methodology

Formal Verification
Exhaustive property checking using formal tools

Simulation (Directed)
Handcrafted tests for known scenarios

Constrained Random + Coverage
CRV guided by functional coverage goals

UVM Testbench
Reusable, layered UVM environment architecture

Stimulus, Checkers, Coverage
What drives, what checks, what measures progress

Assertion-Based Verification
SVA properties bound to design signals

③ When & Priority?

Schedule, levels, and feature priority

Block Level Plans
Unit-level verification before integration

Subsystem Level Plans
Multi-block integration testplan

System Level Plans
Full-chip or SoC-level verification

Feature Priority
P0/P1/P2 ranking — what must pass for tape-out

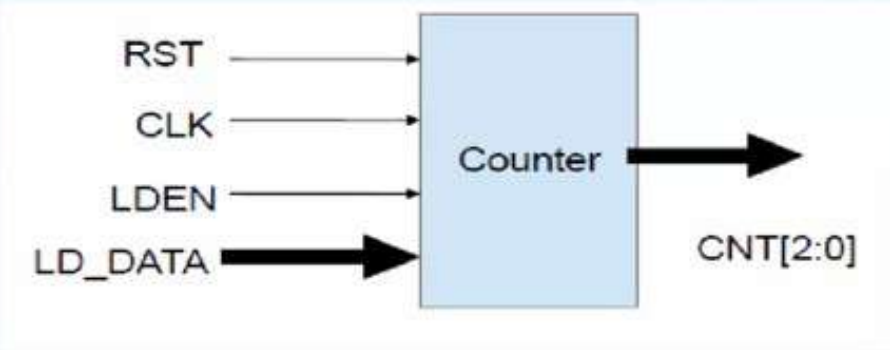
Milestones & Sign-Off
Tied to RTL freeze, tape-out dates

Coverage Closure Gate
Min coverage % required before each milestone

Test Case – Example: 3-Bit Counter

Load Functionality

3-Bit Counter DUV



⚠ Prerequisites for ALL Sequential Test Cases

CLK	Clock must be generated and toggling before test begins
RST	Reset must be asserted/deasserted to initialize DUT state

Signal	Width	Direction	Description
RST	1b	Input	Synchronous reset — sets CNT to 0
CLK	1b	Input	Clock — all ops on posedge
LDEN	1b	Input	Load enable — triggers LD_DATA load
LD_DATA	3b	Input	Data to load into counter
CNT	3b	Output	Counter output — 0 to 7 (wraps)

Test Cases — Load Functionality

TC-01 | Load Random Data — *Basic*

1. Set LD_DATA to a random value (e.g. 5)
2. Assert LDEN = 1 for 1 clock cycle
3. Deassert LDEN = 0
4. Check: CNT[2:0] == LD_DATA on next posedge CLK

TC-02 | Load During Reset — *Basic*

1. Assert RST = 1 and LDEN = 1 simultaneously
2. Apply LD_DATA = 5
3. Check: CNT = 0 (RST has higher priority than LDEN)

TC-03 | Load Max Value (Boundary) — *Corner*

1. Set LD_DATA = 7 (max 3-bit value)
2. Assert LDEN = 1
3. Check: CNT = 7
4. On next cycle (LDEN=0): CNT increments to 0 (wrap)

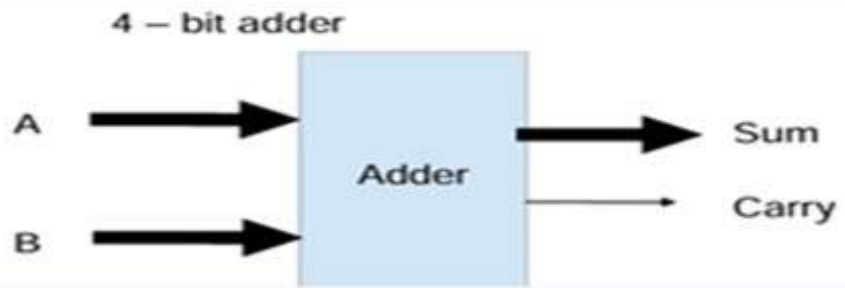
TC-04 | Load While Counter Running

1. Let counter increment freely for N cycles
2. Mid-count: apply LD_DATA = 3, assert LDEN
3. Check: CNT immediately loads 3, resumes counting from 3

Test Case Example – A 4-Bit Adder

Combinational Logic Verification

4-Bit Adder DUV — Combinational Block



✓ Combinational — No CLK or RST Required

Output (Sum, Carry) is purely a function of current inputs A and B — no clock dependency.

Signal	Width	Dir	Description
A	4b	In	First 4-bit operand
B	4b	In	Second 4-bit operand
Sum	4b	Out	Lower 4 bits of A+B result
Carry	1b	Out	Overflow bit — set when A+B > 15

? **What other combinations can we verify?**

Random values, all-zeros, max, alternating patterns, overflow checks...

Test Case Table — 4-Bit Adder

TC#	A	B	Sum	Carry	Type
01	0000	1111	—	0	Basic
02	1111	1111	—	1	Corner
03	1111	0000	1111	0	Basic
04	0000	0000	0000	0	Basic
05	0001	0001	0010	0	Basic
06	1010	0101	1111	0	Basic
07	1000	1000	0000	1	Corner
08	0101	1010	1111	0	Rand
09	1100	0011	1111	0	Rand
10	0110	1001	1111	0	Rand

Module Level Verification

Types of Test Cases

Module Level Verification focuses on verifying individual RTL modules using structured test cases. The choice of test type depends on coverage goals, feature complexity, and the phase of the verification lifecycle.

Types of Test Cases

1 Directed Tests

Tests only one particular feature with fixed test vectors. Handcrafted by the engineer for a known scenario.

- *Early verification phase — sanity and feature-level checks*

2 Constrained Random Tests

Some vectors are randomised within allowed limits (called constraints). Increases coverage by exploring unexpected combinations.

- *Middle phase — coverage-driven random exploration*

3 Corner Cases

Tests combinations of vectors which will occur once in a while. Not easily identified and usually done towards end of the life-cycle.

- *Late phase — after basic and random coverage is achieved*

4 Error Cases

Negative tests which test critical conditions which might cause the DUT to fail. Ensures robust error handling.

- *Throughout — especially important for safety-critical designs*

Comparison & When to Use

Test Type	Coverage	Effort	Phase
Directed	Low	High	Early
CRT	High	Medium	Middle
Corner	Medium	High	Late
Error	Medium	Medium	All

Verification Lifecycle — Test Type Usage

Block Sanity → Directed Tests → CRT + Coverage → Corner + Error → Sign-Off

Additional Test Type

5 Regression Tests

A curated set of previously passing tests re-run after every RTL change to catch regressions. Ensures new changes don't break existing verified functionality.

IP / Module Level Verification

Regression · Coverage · Sign-Off

1 Regression

Running all tests with randomised seeds

The process of running all coded tests with different seeds in order to randomly generate scenarios. The output of regression is functional and code coverage.

- **Seed Variation**
  Different seeds produce different random stimulus combinations, increasing scenario diversity
- **Pass/Fail Tracking**
  Each seed run is logged — any failure triggers a debug cycle before the next regression

Regression Sign-Off Gate

All tests must pass across N seeds before coverage metrics are evaluated for sign-off

2 Functional Coverage

Metric for planned test vector completion

Tells how many of the planned test vectors have been generated. It is a percentage number and all test-benches are required to generate 100% of planned test vectors.

- **Coverage Points**
  Defined in verification plan — each feature maps to one or more coverage bins
- **100% Target**
  Testbenches must hit every planned cover group and bin before sign-off is granted

Coverage Closure Flow

Unexercised bins drive new directed or constrained-random tests until 100% is achieved

3 Code Coverage

Metric for RTL code exercised by tests

Tells how much of the DUT has been exercised in terms of in/out toggling, logical statements, FSMs. This is a %number — may not necessarily reach 100% but is expected to reach the target.

- **Coverage Types**
  Line, branch, toggle, FSM state/arc, condition/expression coverage
- **Exclusions Allowed**
  Unreachable code or hardware tie-offs may be formally excluded with justification

Combined Coverage Gate

Sign-off requires both functional AND code coverage to meet agreed thresholds

Code Coverage — 3-Bit Counter Example

Line · Branch · Toggle · FSM

counter.v — RTL Design (annotated with coverage)

```
1  module COUNTER (
2    input  RST, CLK, LDEN,
3    input  [2:0] LD_DATA,
4    output reg [2:0] CNT
5  );
6  always @(posedge CLK) begin
7    if (RST) begin
8      CNT <= 3'b000;
9    end else if (LDEN) begin
10     CNT <= LD_DATA;
11   end else begin
12     CNT <= CNT + 1;
13   end
14 end
15 endmodule
```

Line hit · Branch hit · Miss (not hit)

Coverage Metrics — What Gets Measured

Line Coverage — 100%

Every executable line in counter.v hit?

Lines 6–12 all executed when RST, LDEN, and increment are driven

▶ *TC-01 (RST), TC-02 (LDEN), TC-03 (increment)*

Branch Coverage — 100%

Every if/else branch taken at least once?

3 branches: RST taken, LDEN taken, else (increment) taken

▶ *All 3 branches need 3 separate test cases to cover*

Toggle Coverage — ~90%

Every signal toggled 0→1 and 1→0?

RST, CLK, LDEN, LD_DATA[2:0], CNT[2:0] must all toggle both ways

▶ *CNT[2] may not toggle if counter never reaches 4+ (load values needed)*

FSM Coverage — N/A

Not applicable — counter has no explicit FSM

Counter is described as a sequential datapath, not a state machine

▶ *FSM coverage applies when design contains state register + next-state logic*

Functional Coverage — 3-Bit Counter Example

Covergroup · Coverpoint · Bins

counter_coverage.sv — Functional Coverage Model

```
// Covergroup for 3-bit counter
covergroup counter_cg @(posedge CLK);

  // Coverpoint 1: CNT output values
  cp_cnt: coverpoint CNT {
    bins zero     = {3'b000};                BIN
    bins low      = {[3'b001:3'b011]};       BIN
    bins high     = {[3'b100:3'b110]};       BIN
    bins max      = {3'b111};                BIN
  }

  // Coverpoint 2: Control signals
  cp_rst:  coverpoint RST;                   CP
  cp_lden: coverpoint LDEN;                  CP

  // Cross coverage: RST x LDEN
  cx_ctrl: cross cp_rst, cp_lden;            CX

```

BIN — Coverage bin CP — Coverpoint CX — Cross coverage

Coverage Results — Bins & Hit Status

cp_cnt — Counter Output Bins

Bin	Range	Meaning	Hit?	Test
zero	000	Counter at 0 (reset state)	✓	RST assertion
low	001–011	Lower half (1–3)	✓	Free-run 1–3 cycles
high	100–110	Upper half (4–6)	✓	Free-run 4–6 cycles
max	111	Max value — wrap check	✗	Load 7 or run to wrap

cp_cnt Coverage: 3 of 4 bins hit → 75% — 75%

cp_rst & cp_lden — Control Signal Coverpoints

cp_rst	RST=0 ✓	RST=1 ✓
cp_lden	LDEN=0 ✓	LDEN=1 ✓

cx_ctrl — Cross Coverage: RST × LDEN (4 bins)

✓ **RST=0, LDEN=0** *Free-run increment*	✓ **RST=1, LDEN=0** *Reset only*
✓ **RST=0, LDEN=1** *Load data scenario*	✗ **RST=1, LDEN=1** *Simultaneous — corner case!*

Overall Functional Coverage:
→ Hit 'max' bin & RST+LDEN corner case to reach 100%

cp_cnt	cp_ctrl	cx_ctrl
75%	100%	75%

What Is a Bug?

Blocking vs Non-Blocking Assignment

A Bug is a defect, mistake, flaw or error in a design that prevents it from behaving as intended. A common RTL bug: using **blocking (=) instead of non-blocking (<=) assignments** in sequential always blocks.

✗ BUGGY Code — Blocking Assignment (=)

```
always @(posedge clk)
begin
  b = a;    // blocking!
  c = b;    // uses NEW b (= a)
end
```

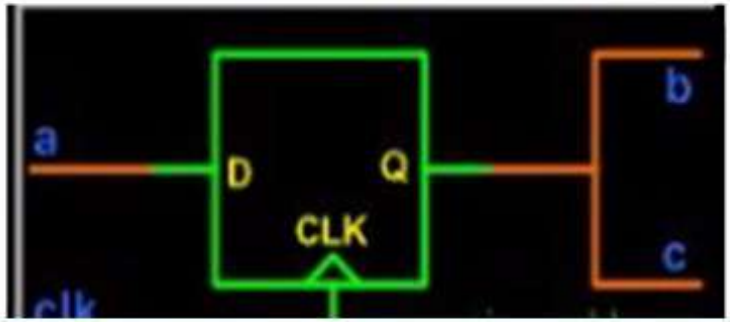
⚠ What goes wrong:
b gets a's value immediately. Then c=b copies the NEW b — so c and b both become a in the same clock cycle. Only 1 FF instead of 2 in the pipeline!

✓ CORRECT Code — Non-Blocking Assignment (<=)

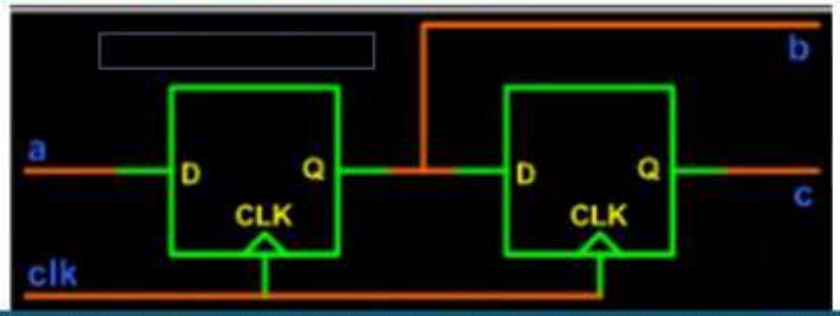
```
always @(posedge clk)
begin
  b <= a;   // schedules update
  c <= b;   // uses OLD b
end
```

✓ Both b and c update on the SAME clock edge using OLD values → 2 pipeline FFs

Circuit — Only 1 Flip-Flop (Bug!)



Circuit — 2 Flip-Flops (Pipeline — Correct!)



Rule: Always use <= (non-blocking) in sequential always @(posedge clk) blocks

Debugging

Bug Detection at Pin Level

Debugging is the process of identifying bugs in the RTL design. A bug is any condition that causes the design to behave differently than expected. During verification, **bugs must be detected at the pin level** — observable at inputs/outputs of the DUT.

Debugging — Key Concepts

1. **What is Debugging?**
   Process of identifying and locating bugs in the RTL design after a test failure is observed

2. **What is a Bug?**
   Any condition in the RTL that causes the design to behave differently than the design spec intends

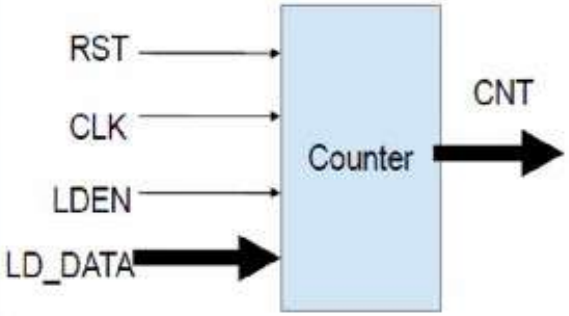
3. **Pin-Level Detection**
   Bugs must be detected at I/O pins of the DUT — the testbench observes only external signals, not internal wires

4. **Bug vs. Testbench Issue**
   Failures can be either a design bug or a testbench error — both must be systematically ruled out

5. **Root Cause Analysis (NEW)**
   Waveform debugging, assertions, and $display/$monitor help trace from pin-level symptom to root cause

+ Point added beyond original slide

Counter DUT — Bug Example: Priority Coding Error



If, upon asserting RST and LDEN at the same time, CNT does not become 0, then this is a bug and it is due to incorrect coding as shown

```
// Verilog code for counter

module COUNTER (RST,CLK,CNT,LDEN, LD_DATA);

  input RST,CLK,LDEN;
  input [2:0] LD_DATA
  output reg [2:0] CNT;

  always @ (posedge CLK )
    if (LDEN)
       CNT <= LD_DATA;
    else if(RST)
       CNT <= 0;
    else
   CNT <= CNT + 1;

endmodule
```

LDEN has Higher priority than RST

Bug: Code checks LDEN before RST — so RST+LDEN asserted simultaneously loads data instead of resetting. RST should have highest priority. Correct order: `if(RST) ... else if(LDEN)`